Clean Code Principles

SOLID, DRY, KISS, YAGNI, LoD & more — with Java Examples

March 2026

1. Introduction

Clean code is code that is easy to read, understand, change, and maintain. Principles like SOLID, DRY, and KISS are distilled wisdom from decades of software engineering practice — they are not arbitrary rules but patterns that consistently lead to more maintainable systems.

This document covers the most important coding principles, explaining each one, showing a concrete Java example of code that violates it, then a corrected version.

- SOLID — five OOP design principles (Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion)
- DRY — Don't Repeat Yourself
- KISS — Keep It Simple, Stupid
- YAGNI — You Aren't Gonna Need It
- LoD — Law of Demeter (Principle of Least Knowledge)
- Fail Fast — surface errors early and loudly
- Composition over Inheritance

2. SOLID Principles

The SOLID acronym was coined by Robert C. Martin ("Uncle Bob"). Each letter is an independent principle; together they guide towards loosely coupled, highly cohesive object-oriented design.

2.1 S — Single Responsibility Principle (SRP)

"A class should have only one reason to change."

Each class should do one thing and own that thing completely. If a class changes for two different reasons, it has two responsibilities and should be split.

Before (violation)

```
// BAD: UserService does BOTH business logic AND email formatting
public class UserService {
public void registerUser(String email, String password) {
// 1. Save to DB
db.save(new User(email, password));
```

```
// 2. Format and send email <-- second responsibility
String html = "<h1>Welcome " + email + "</h1>";
emailClient.send(email, "Welcome!", html);
}
}
```

After (principle applied)

```
// GOOD: split into focused classes
public class UserService {
private final UserRepository repo;
private final EmailService email;
public void registerUser(String e, String pwd) {
repo.save(new User(e, pwd));
email.sendWelcome(e);
}
}
public class EmailService {
public void sendWelcome(String address) {
String html = "<h1>Welcome " + address + "</h1>";
emailClient.send(address, "Welcome!", html);
}
}
```

A good heuristic: if you struggle to name a class with a single noun, it probably has too many responsibilities.

2.2 O — Open/Closed Principle (OCP)

"Software entities should be open for extension, closed for modification."

Add new behaviour by adding new code (a new class or implementation), not by editing existing, tested code. Typical tool: polymorphism and interfaces.

Before (violation)

```
// BAD: every new shape requires modifying AreaCalculator
public class AreaCalculator {
public double calculate(Object shape) {
```

```
if (shape instanceof Circle c) return Math.PI * c.r * c.r;
if (shape instanceof Rectangle r) return r.w * r.h;
// Need triangle? Edit this class again...
throw new IllegalArgumentException("Unknown shape");
}
}
```

After (principle applied)

```
// GOOD: each shape knows its own area
public interface Shape {
double area();
}
public record Circle(double r) implements Shape { public double area() { return Math.PI * r * r; } }
public record Rectangle(double w, double h) implements Shape { public double area() { return w * h; } }
public record Triangle(double b, double h) implements Shape { public double area() { return 0.5*b*h; } }
// AreaCalculator never changes
public class AreaCalculator {
public double total(List<Shape> shapes) {
return shapes.stream().mapToDouble(Shape::area).sum();
}
}
```

OCP does not mean never edit code. It means stable, tested behaviour should not be disturbed when adding new features.

2.3 L — Liskov Substitution Principle (LSP)

"Subtypes must be substitutable for their base types without altering program correctness."

If S extends T, you must be able to use an S wherever a T is expected and the program should still behave correctly. Violations often appear when a subclass throws exceptions a caller doesn't expect, or silently ignores a method.

Before (violation)

```
// BAD: Square "is a" Rectangle — but breaks the invariant
```

```
class Rectangle {
protected int width, height;
public void setWidth(int w) { this.width = w; }
public void setHeight(int h) { this.height = h; }
public int area() { return width * height; }
}
class Square extends Rectangle {
@Override public void setWidth(int w) { width = height = w; } // breaks caller expectations!
@Override public void setHeight(int h) { width = height = h; }
}
// Caller assumes setting width won't change height
Rectangle r = new Square();
r.setWidth(5); r.setHeight(3);
System.out.println(r.area()); // 9, but caller expected 15
```

After (principle applied)

```
// GOOD: prefer composition / separate abstractions
interface Shape { int area(); }
record Rectangle(int w, int h) implements Shape { public int area() { return w * h; } }
record Square(int side) implements Shape { public int area() { return side * side; } }
// No inheritance between Rectangle and Square — no substitution problem
```

If a subclass needs to override a method and do nothing (or throw), that is a red flag for an LSP violation.

2.4 I — Interface Segregation Principle (ISP)

"No client should be forced to depend on methods it does not use."

Prefer small, focused interfaces over large "fat" ones. Clients that implement your interface should only need to implement relevant methods.

Before (violation)

```
// BAD: one fat interface forces RobotWorker to implement eat()
interface Worker {
```

```
void work();
void eat(); // robots don't eat!
void sleep(); // robots don't sleep!
}
class RobotWorker implements Worker {
public void work() { /* do work */ }
public void eat() { throw new UnsupportedOperationException(); }
public void sleep() { throw new UnsupportedOperationException();
}
}
```

After (principle applied)

```
// GOOD: segregated interfaces
interface Workable { void work(); }
interface Feedable { void eat(); }
interface Restable { void sleep();}
class HumanWorker implements Workable, Feedable, Restable {
public void work() { System.out.println("Working..."); }
public void eat() { System.out.println("Eating..."); }
public void sleep() { System.out.println("Sleeping...");}
}
class RobotWorker implements Workable {
public void work() { System.out.println("Processing..."); }
// no eat/sleep — no problem
}
```

Java's own standard library follows ISP: Comparable, Iterable, Runnable are all single-method interfaces.

2.5 D — Dependency Inversion Principle (DIP)

"Depend on abstractions, not concretions."

High-level policy modules should not import low-level detail modules. Both should depend on interfaces. This is the foundation for dependency injection (DI) frameworks like Spring.

Before (violation)

```
// BAD: OrderService is tightly coupled to MySQLRepository
public class OrderService {
private MySQLOrderRepository repo = new MySQLOrderRepository(); // hard dependency
public void placeOrder(Order order) {
repo.save(order);
}
}
```

After (principle applied)

```
// GOOD: depend on the interface
public interface OrderRepository {
void save(Order order);
}
public class MySQLOrderRepository implements OrderRepository {
public void save(Order order) { /* MySQL logic */ }
}
public class OrderService {
private final OrderRepository repo; // inject the abstraction
public OrderService(OrderRepository repo) { this.repo = repo; }
public void placeOrder(Order order) { repo.save(order); }
}
// Swap implementations without touching OrderService
OrderService svc = new OrderService(new MySQLOrderRepository());
OrderService test = new OrderService(new InMemoryOrderRepository());
```

Constructor injection (as above) is the preferred DI style — dependencies are explicit and the class is easily testable.

3. DRY — Don't Repeat Yourself

3.1 The Principle

"Every piece of knowledge must have a single, unambiguous, authoritative representation."

Duplicated code is duplicated risk: fix a bug in one copy and forget the other. DRY is about knowledge duplication, not just literal copy-paste. Two identical algorithms that express different business rules are NOT a DRY violation.

Before (violation)

```
// BAD: tax calculation duplicated in two places
public double getCartTotal(List<Item> items) {
double subtotal = items.stream().mapToDouble(Item::price).sum();
return subtotal + subtotal * 0.08; // 8% tax
}
public double getInvoiceTotal(List<Item> items) {
double subtotal = items.stream().mapToDouble(Item::price).sum();
return subtotal + subtotal * 0.08; // duplicated tax logic
}
```

After (principle applied)

```
private static final double TAX_RATE = 0.08;
private double withTax(double subtotal) {
return subtotal * (1 + TAX_RATE);
}
public double getCartTotal(List<Item> items) {
return withTax(items.stream().mapToDouble(Item::price).sum());
}
public double getInvoiceTotal(List<Item> items) {
return withTax(items.stream().mapToDouble(Item::price).sum());
}
```

The rule of three: the first time you write something, just write it. The second time, note the duplication. The third time, refactor.

4. KISS — Keep It Simple, Stupid

4.1 The Principle

"Most systems work best if they are kept simple. Simplicity should be a key goal."

Complex code is hard to read, hard to test, and a breeding ground for bugs. Before reaching for a clever solution, ask whether a simpler one would do. Prefer readability over cleverness.

Before (violation)

```
// BAD: over-engineered reversal
public String reverseWords(String sentence) {
return IntStream.range(0, sentence.split(" ").length)
.map(i -> sentence.split(" ").length - 1 - i)
.mapToObj(i -> sentence.split(" ")[i])
.collect(Collectors.joining(" "));
}
```

After (principle applied)

```
// GOOD: straightforward and immediately understandable
public String reverseWords(String sentence) {
String[] words = sentence.split(" ");
Collections.reverse(Arrays.asList(words));
return String.join(" ", words);
}
```

"Debugging is twice as hard as writing the code in the first place. If you write the code as cleverly as possible, by definition you are not smart enough to debug it." — Brian Kernighan

5. YAGNI — You Aren't Gonna Need It

5.1 The Principle

"Always implement things when you actually need them, never when you just foresee that you might need them."

Speculative generality adds code complexity, increases maintenance burden, and often turns out to be wrong. Build for today's requirements; refactor when tomorrow's are clear.

Before (violation)

```
// BAD: adding a plugin system no one asked for
public class ReportGenerator {
private List<ReportPlugin> plugins = new ArrayList<>();
private PluginRegistry registry;
```

```
private ReportConfig config;
// 200 lines of plugin infrastructure that nobody uses yet...
public void registerPlugin(ReportPlugin p) { plugins.add(p); }
public void generateReport(Data data) {
plugins.forEach(p -> p.preProcess(data));
// actual one-liner report logic buried here
plugins.forEach(p -> p.postProcess(data));
}
}
```

After (principle applied)

```
// GOOD: just generate the report
public class ReportGenerator {
public String generate(Data data) {
return data.rows().stream()
.map(Row::toCsvLine)
.collect(Collectors.joining("\n"));
}
}
// Add plugin system IF AND WHEN a real requirement arrives
```

YAGNI is not an excuse to write bad, un-extensible code. Good abstractions (SRP, DIP) naturally enable extension without pre-building features.

6. Law of Demeter (Principle of Least Knowledge)

6.1 The Principle

"Talk only to your immediate friends; don't talk to strangers."

A method should only call methods on: itself, its fields, its parameters, objects it creates, and globally accessible objects. Long chains like a.getB().getC().doSomething() are a violation — you are reaching through objects you don't own.

Before (violation)

```
// BAD: OrderPlacer reaches deep into customer's internals
public class OrderPlacer {
public void placeOrder(Order order) {
```

```
String city = order.getCustomer().getAddress().getCity();
if (city.equals("NYC")) applyNycTax(order);
}
}
```

After (principle applied)

```
// GOOD: delegate to Order; Order knows its customer
public class Order {
public String getCustomerCity() {
return customer.getAddress().getCity();
}
}
public class OrderPlacer {
public void placeOrder(Order order) {
if (order.getCustomerCity().equals("NYC")) applyNycTax(order);
}
}
```

A tell-tale sign of a LoD violation: adding a field to a deeply nested object forces changes to unrelated classes.

7. Fail Fast

7.1 The Principle

"Report errors as close to their source as possible."

Validate inputs early. Throw an exception (or use a validation library) the moment you detect something is wrong — do not silently continue with bad data only to crash somewhere opaque later.

Before (violation)

```
// BAD: null sneaks through and crashes far from the source
public class OrderService {
public void processOrder(Order order) {
// ... 50 lines later ...
order.getCustomer().sendConfirmation(); // NullPointerException
here
}
```

}

After (principle applied)

// GOOD: fail at the entry point with a clear message

public class OrderService {

public void processOrder(Order order) {

Objects.requireNonNull(order, "order must not be null");

Objects.requireNonNull(order.getCustomer(), "order.customer must not be null");

// all following code can trust order is valid

order.getCustomer().sendConfirmation();

}

}

Java's Objects.requireNonNull, Guava's Preconditions, and Jakarta Bean Validation (@NotNull, @Size) are your friends.

8. Composition over Inheritance

8.1 The Principle

"Favour object composition over class inheritance."

Deep inheritance hierarchies create tight coupling, fragile base-class problems, and restricted reuse. Composition — assembling behaviours from small collaborating objects — is more flexible and easier to test.

Before (violation)

// BAD: brittle hierarchy — what if Dog also needs to Swim?

class Animal { void breathe() { /*...*/ } }

class Walkable extends Animal { void walk() { /*...*/ } }

class Swimmer extends Animal { void swim() { /*...*/ } }

// Dog needs both walk AND swim — Java single inheritance blocks this cleanly

class Dog extends Walkable { /* can't also extend Swimmer */ }

After (principle applied)

// GOOD: compose behaviours

interface Walkable { void walk(); }

interface Swimmable { void swim(); }

```
class WalkImpl implements Walkable { public void walk() { System.out.println("Walking"); } }

class SwimImpl implements Swimmable { public void swim() { System.out.println("Swimming"); } }

class Dog {

private final Walkable walker = new WalkImpl();

private final Swimmable swimmer = new SwimImpl();

public void walk() { walker.walk(); }

public void swim() { swimmer.swim(); }

}

// Trivial to add Flying later: private final Flyable flyer = new FlyImpl();
```

Inheritance is "is-a"; composition is "has-a". When in doubt, prefer has-a — you can always change the composed object at runtime.

9. Quick-Reference Cheat Sheet

Principle	In one sentence	Violation smell	F
SRP	One class, one reason to change	Class name has 'And' or 'Manager'	S
OCP	Extend by adding, not editing	switch/if-chains growing with new types	U
LSP	Subtype behaviour must not surprise callers	Subclass throws UnsupportedOperation	F
ISP	Small focused interfaces over fat ones	Empty/throwing interface methods	S
DIP	Depend on abstractions	new ConcreteClass() inside a service	I
DRY	One source of truth for each piece of knowledge	Same logic copy-pasted across methods	E
KISS	Prefer the simpler solution	Can't explain it in one sentence	F
YAGNI	Only build what is needed now	Infrastructure nobody uses yet	D
LoD	Don't reach through objects you don't own	a.getB().getC().doX() chains	A
Fail Fast	Report errors immediately at their source	NullPointerException far from null site	V
Composition	Prefer has-a over is-a	Deep inheritance trees	E

10. Further Reading

Clean Code — Robert C. Martin (2008) — the accessible companion to SOLID

The Pragmatic Programmer — Hunt & Thomas (20th anniversary ed. 2019) — DRY, KISS, YAGNI in depth

Effective Java (3rd ed.) — Joshua Bloch (2018) — item-by-item Java best practices

Refactoring: Improving the Design of Existing Code — Martin Fowler (2nd ed. 2018)